252-0027

Einführung in die Programmierung

Übungen

Woche 9: Klassen, Verlinkte Objekte

Joran Bühler
Departement Informatik
ETH Zürich

Probleme Lösen

Probleme Lösen: Labyrinth

Aufgabe 1: Labyrinth (2021 W8)

Ein Labyrinth besteht aus einer Menge von Räumen, welche durch die Klasse `Room` dargestellt werden. Die Klasse hat zwei Attribute: Der Integer `age` (grösser gleich 0) beschreibt das Alter des Raums und der Array `doorsTo` (nie `null`) beschreibt die Türen von diesem Raum zu anderen Räumen. Alle Türen sind Falltüren, d.h. sie funktionieren nur in eine Richtung. Ein Raum ist ein Ausgang aus dem Labyrinth, wenn keine Türen von dem Raum wegführen, das heisst, wenn `doorsTo` eine Länge von 0 hat.

Für alle Aufgaben werden Sie in einen zufälligen Raum geworfen, welcher als Argument gegeben wird (garantiert nicht `null`) und von welchem aus Sie die Aufgabe lösen müssen. Sie dürfen für alle Aufgaben annehmen, dass es im Labyrinth keinen Zyklus gibt. Das heisst, dass man einen Raum, welchen man durch eine Türe verlassen hat, nie wieder erreichen kann indem man weiteren Türen folgt. Eine Sequenz von N Räumen $r_1, \ldots, r_N$ ist ein *Lösungspfad* für einen Raum `room` genau dann wenn: (1) Der erste Raum r_1 ist der Raum `room`, (2) der letzte Raum r_N ist ein Ausgang, und (3) jeder Raum r_i mit $1 \leq i < N$ hat eine Tür zum nächsten Raum in der Sequenz r_{i+1}.

Probleme Lösen: Labyrinth

1. Implementieren Sie die Methode `Labyrinth.task1(Room room)`. Die Methode soll true zurückgeben genau dann, wenn es einen Lösungspfad $r_1, \ldots, r_N$ für `room` gibt, sodass:
   - Für jede Teilsequenz $r_1, ..., r_i$ mit $1 \leq i \leq N$ gilt, dass die Summe der Alter der Räume $r_1, ..., r_i$ nicht durch 3 teilbar ist.
2. Implementieren Sie die Methode `Labyrinth.task2(Room room)`. Die Methode soll true zurückgeben genau dann, wenn es zwei Lösungspfade $r_1, \ldots, r_N$ und $s_1, \ldots, s_N$ für `room` gibt, sodass:
   - Die Räume r_i und s_i haben das gleiche Alter für jedes i mit $1 \leq i \leq N$.
   - Für mindestens ein i mit $1 \leq i \leq N$ gilt, dass r_i und s_i unterschiedlich sind (verschiedene Referenzen).

Wichtig!

Sie dürfen Methoden und Felder der Klasse `Room` hinzufügen. Tests finden Sie in der Datei "LabyrinthTest.java". **Tipp:** Lösen Sie die Aufgaben rekursiv. Für keine der Aufgaben müssen Sie alle Pfade generieren und dann erst prüfen, dass die Eigenschaften gelten. Manche der Tests enthalten Labyrinthe mit einer extrem grossen Anzahl an Pfaden aber leichten Lösungen.

Organisatorisches

- Website: **https://joranbu.github.io/eprog/**



Kahoot

https://create.kahoot.it/details/u8-eprog-shortened-klassen-basics-loop-invarianten/0941ad27-5d84-4aff-884f-4e0f52d6b607?drawer=

Vorbesprechung

Aufgabe 1: Loop-Invariante



Gegeben ist eine Postcondition für das folgende Programm

```
public int compute(String s, char c) {
  // Precondition s != null
  int x;
  int n;

  x = 0;
  n = 0;

  // Loop Invariante:
  while (x < s.length()) {
    if (s.charAt(x) == c) {
      n = n + 1;
    }
    x = x + 1;
  }

  // Postcondition: count(s, c) == n
  return n;
}
```

Die Methode `count(String s, char c)` gibt zurück wie oft der Character `c` im String `s` vorkommt. Schreiben Sie die Loop Invariante in die Datei "LoopInvariante.txt". **Tipp:** Benutzen Sie die `substring` Methode.

Aufgabe 2: Linked List



Bisher haben Sie Arrays verwendet, wenn Sie mit einer grösseren Anzahl von We eitet haben. Ein Nachteil von Arrays ist, dass die Grösse beim Erstellen des Arrays fe legt den muss und danach nicht mehr verändert werden kann. In dieser Aufgabe implem tieren Sie se st eine Datenstruktur, bei welcher die Grösse im Vornherein nicht bestimmt ist und welche jederzeit wachsen und schrumpfen kann: eine *linked list* oder *verkettete Liste*.

Eine verkettete Liste besteht aus mehreren Objekten, welche Referenzen zueinander haben. Für diese Aufgabe besteht jede Liste aus einem "Listen-Objekt" der Klasse `LinkedIntList`, welches die gesamte Liste repräsentiert, und aus mehreren "Knoten-Objekten" der Klasse `IntNode`, eines für jeden Wert in der Liste. Die Liste heisst "verkettet", weil jedes Knoten-Objekt ein Feld mit einer Referenz zum nächsten Knoten in der Liste enthält. Das `LinkedIntList`-Objekt schliesslich enthält eine Referenz zum ersten und zum letzten Knoten und hat ausserdem ein Feld für die Länge der Liste.

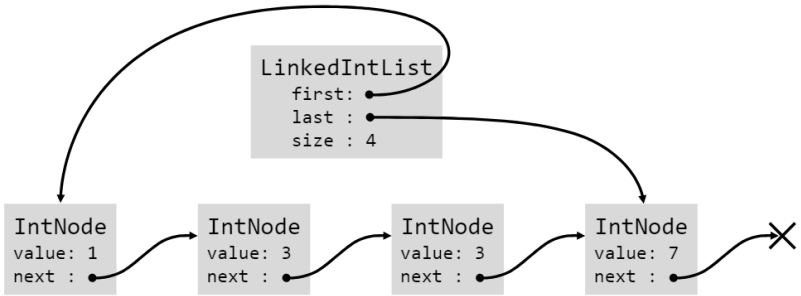


Abbildung 1: Verkettete Liste mit Werten 1, 3, 3, 7.

Aufgabe 2: Linked List

Name	Parameter	Rückg.-Typ	Beschreibung
`addLast`	`int value`	`void`	fügt einen Wert am Ende der Liste ein
`addFirst`	`int value`	`void`	fügt einen Wert am Anfang der Liste ein
`removeFirst`		`int`	entfernt den ersten Wert und gibt ihn zurück
`removeLast`		`int`	entfernt den letzten Wert und gibt ihn zurück
`clear`		`void`	entfernt alle Wert in der Liste
`isEmpty`		`boolean`	gibt zurück, ob die Liste leer ist
`get`	`int index`	`int`	gibt den Wert an der Stelle `index` zurück
`set`	`int index, int value`	`void`	ersetzt den Wert an der Stelle `index` mit `value`
`getSize`		`int`	gibt zurück, wie viele Werte die Liste enthält

Einige dieser Methoden dürfen unter gewissen Bedingungen nicht aufgerufen werden. Zum Beispiel darf `removeFirst()` nicht aufgerufen werden, wenn die Liste leer ist, oder `get()` darf nicht aufgerufen werden, wenn der gegebene Index grösser oder gleich der aktuellen Länge der Liste ist. In solchen Situationen soll sich Ihr Programm mit einer Fehlermeldung beenden. Verwenden Sie folgendes Code-Stück dafür:

```
if(condition) {
    Errors.error(message);
}
```

Ersetzen Sie *`condition`* mit der Bedingung, unter welcher das Programm beendet werden soll, und *`message`* mit einer hilfreichen Fehlermeldung. Die `Errors`-Klasse befindet sich bereits in Ihrem Projekt, aber Sie brauchen sie im Moment nicht zu verstehen.



Aufgabe 3: Executable Graph

In dieser Aufgabe verwenden wir gerichtete azyklische Graphen, um Programme zu repräsentieren. Der Programmzustand ist dabei immer durch ein Tupel $(sum, counter)$ gegeben, wobei *sum* und *counter* ganze Zahlen sind. Programmzustände werden durch `ProgramState`-Objekte modelliert, wobei `ProgramState.getSum()` (bzw. `ProgramState.getCounter()`) dem ersten Element (bzw. dem zweiten Element) des Tupels entspricht.

Eine Ausführung des Programms manipuliert den Programmzustand und das Resultat eines Programms ist gegeben durch den erreichten Programmzustand, nachdem alle Operationen im Programm ausgeführt wurden. Programme können nichtdeterministisch sein: Das heisst, für ein einzelnes Programm kann es für den gleichen Startzustand mehrere Programmausführungen geben, welche zu unterschiedlichen Resultaten führen.

Knoten in Graphen werden durch `Node`-Objekte modelliert. `Node.getSubnodes()` gibt die Kinderknoten als ein Array zurück (m ist genau dann ein Kinderknoten von n, wenn es eine ausgehende gerichtete Kante von n zu m gibt). Wir unterscheiden drei Arten von Knoten, wobei die Methode `Node.getType()` die Knotenart als `String` zurückgibt. Um ein Programm, welches durch den Knoten `n` repräsentiert wird, auszuführen, muss man den "Knoten `n` ausführen". Wir beschreiben nun die drei Knotenarten und jeweils die Ausführung der Knoten:

Aufgabe 3: Executable Graph

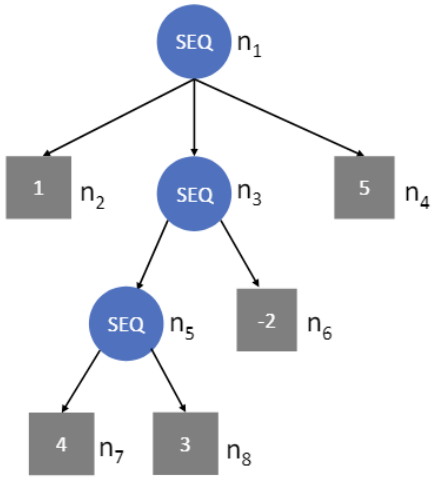
1. Additionsknoten (`Node.getType()` ist "ADD"): Solche Knoten besitzen einen Additionswert a gegeben durch `Node.getValue()` (eine ganze Zahl) und bei der Ausführung dieses Knotens wird der Programmzustand von $(sum, counter)$ zu $(sum + a, counter + 1)$ aktualisiert. Die Kinderknoten von solchen Knoten werden bei der Ausführung ignoriert.

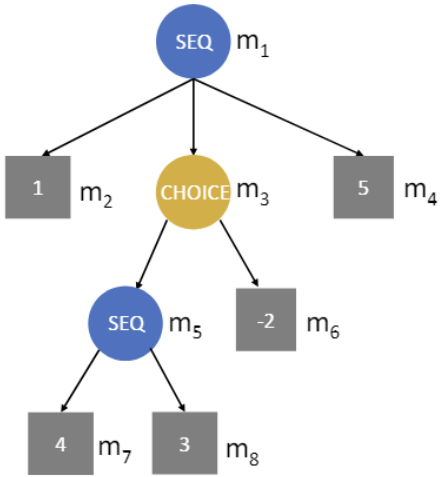
2. Sequenzknoten (`Node.getType()` ist "SEQ"): Bei der Ausführung eines Sequenzknoten `n` werden die Kinderknoten von `n` nacheinander ausgeführt. Die Reihenfolge in welcher die Kinderknoten ausgeführt werden spielt keine Rolle, da der erreichte Programmzustand für jede Reihenfolge gleich ist. `Node.getValue()` ist irrelevant.

3. Auswahlknoten (`Node.getType()` ist "CHOICE"): Bei der Ausführung eines Auswahlknoten `n` wird ein beliebiger Kinderknoten von `n` ausgewählt und ausgeführt. `Node.getValue()` ist irrelevant. Diese Knoten führen zu Nichtdeterminismus.

Sie dürfen davon ausgehen, dass Sequenz- und Auswahlknoten immer mindestens einen Kinderknoten haben, und dass es zwischen zwei Knoten immer höchstens einen Pfad gibt. Die folgende Abbildung zeigt zwei Beispielgraphen, wobei Knoten mit der Beschriftung "SEQ" (bzw. "CHOICE") Sequenzknoten (bzw. Auswahlknoten) entsprechen und die Zahlen in Additionsknoten den Additionswerten entsprechen.

Aufgabe 3: Executable Graph







Aufgabe 4: Energiespiel

In dieser Aufgabe üben Sie den Umgang mit Enums. Dafür haben Sie einen Ordner `EnergieSpiel` mit drei Klassen `GameApp`, `Game` und `Player`, sowie ein Enum `Character`. Diese sind bereits so implementiert, dass alles funktioniert. Die Klasse `Player` hat jedoch ein Feld `character` von Typ `String`. Java lässt also zu, dass in diesem Feld ein beliebiger String abgespeichert werden kann. Das Spiel hat aber eigentlich nur genau drei Möglichkeiten: `HONEST`, `TRICKSTER` oder `SORCEROR`. Das Enum `Character` mit diesen drei Optionen existiert bereits. Ändern Sie den Typ des Feldes zu `Character` und passen Sie den Code in allen drei Klassen so an, dass die Charakter-Logik überall den Typ `Character` statt `String` verwendet.



Aufgabe 5: Methoden vergleichen

Für diese Aufgabe betrachten wir folgende Methode:

```
public int T(int x, int y) {
    // body
}
```

Anstelle des Kommentars `// body` im Methodenrumpf setzen wir jeweils eine der angegebenen Anweisungsfolgen (a)–(e) ein. Vergleichen Sie für jedes unten aufgeführte Paar die Rückgabewerte für alle `x` und `y`. Antworten Sie in genau einem dieser Formate: `I; Rückgabewert: ...`, falls die Werte *immer* gleich sind; `M; Bedingung: ...; Rückgabewert: ...`, falls sie nur *manchmal* gleich sind; oder `N`, falls sie *nie* gleich sind.

Bei `M` vereinfachen Sie den gemeinsamen Rückgabewert mithilfe der angegebenen Bedingung. Aus der Bedingung `y == 9` und dem Wert `y + 1` wird beispielsweise `Rückgabewert: 10`. Falls der Wert weiterhin von Variablen abhängt, ist der vollständige Java-Ausdruck erforderlich: Für `y == 2 ? 4 : 1` sind `4` oder `1` allein falsch; `y == 9 ? y + 1 : 0` wird als `y == 9 ? 10 : 0` angegeben.

(a)
```
return x < y ? x + 1 : y - 1;
```

(b)
```
if (x >= y) {
    return y - 1;
}
return x + 1;
```